



Universidad Nacional Autónoma de México

Facultad de Ingeniería

Fundamentos de Programación

Ing. Ariel Adara Mercado Martinez

Proyecto Final

Alumnos:

Barrera Hernandez Marcos Omar

Huerta Lopez Angel Sebastian

Plata Ramírez Elian Alejandro

Bitácora de trabajo

Planeación del proyecto

Nos reunimos para definir la idea general del proyecto. La instrucción era desarrollar un videojuego inspirado en Pac-Man, nosotros decidimos utilizar personajes de Pokémon para hacerlo más atractivo visualmente. Analizamos nuestros conocimientos adquiridos y determinamos que implementar variables, arreglos, funciones, ciclos y estructuras condicionales para desarrollar el juego. También diseñamos la estructura de los mapas y la forma en que se moverían el jugador y los enemigos.

Creación de mapas y movimiento del jugador

Comenzamos desarrollando los mapas del juego utilizando matrices de caracteres para representar paredes, espacios libres, puntos y posiciones iniciales de los personajes. Posteriormente implementamos el movimiento del jugador mediante el uso de las teclas de dirección del teclado. Realizamos varias pruebas para verificar que el personaje no pudiera atravesar las paredes y que se desplazara correctamente dentro del laberinto.

Implementación de fantasmas y colisiones

Desarrollamos la lógica de movimiento de los fantasmas utilizando direcciones aleatorias y movimientos orientados hacia la posición del jugador. También implementamos el sistema de colisiones entre el jugador y los fantasmas. Cuando ocurre una colisión, el jugador pierde una vida y todos los personajes regresan a sus posiciones iniciales. Durante esta etapa realizamos numerosas pruebas para corregir errores de movimiento y evitar que los fantasmas atravesaran las paredes del mapa.

Sistema de puntajes y niveles

Implementamos la recolección de pellets dentro del laberinto para aumentar el puntaje del jugador. Cada pellet recolectado suma puntos y desaparece del mapa. Además, desarrollamos un sistema de progresión por niveles. Cuando el jugador recolecta todos los pellets de un nivel, el programa carga automáticamente el siguiente mapa. El juego cuenta con tres niveles diferentes. También agregamos el guardado de puntajes en un archivo de texto para conservar los resultados obtenidos al finalizar cada partida.

Integración de SDL2 y recursos gráficos

Durante esta etapa trabajamos con las bibliotecas SDL2 y SDL_image para crear la ventana del juego y mostrar los elementos gráficos. Fue necesario comprobar constantemente que el proyecto compilara correctamente con SDL2, verificando la correcta instalación de las bibliotecas y la carga de los recursos gráficos. Las animaciones de los personajes Pikachu y Gengar fueron obtenidas con ayuda de herramientas de inteligencia artificial y posteriormente adaptadas para ser utilizadas dentro del juego.

Animaciones y mejoras visuales

Implementamos las animaciones de los personajes mediante el uso de spritesheets. Se configuraron distintos cuadros de animación para representar los movimientos de Pikachu y Gengar en diferentes direcciones. También mejoramos la presentación visual del juego mediante colores para las paredes, pellets y elementos del escenario. Se realizaron pruebas continuas para asegurar que las animaciones se mostraran correctamente durante la ejecución.

Pruebas finales y documentación

Realizamos pruebas completas del proyecto para verificar el funcionamiento de los tres niveles, el sistema de vidas, las colisiones, el conteo de puntos y las animaciones. Se corrigieron errores menores encontrados durante la fase de pruebas y se comprobó nuevamente que el proyecto compilara y ejecutara correctamente utilizando SDL2. Finalmente organizamos el código fuente, los archivos de imágenes y la documentación necesaria para realizar la entrega final del proyecto.

Pseudocódigos y Diagramas de Flujo de Funciones a Usar

Ciclo principal Main

Parte 1: Preparación del Juego

Entrada: Assents (sprites y mapa)

Salida: Resultado de la preparación (las estructuras de datos creadas en memoria y las ventanas físicas que se muestran en pantalla)

INICIO (Parte 1)

ultimoFantasma <- 0 (Reloj inicial para los enemigos)

Iniciar números aleatorios con el tiempo actual (srand)

SI falla la inicialización de video de SDL ENTONCES Mostrar error y salir

SI falla el lector de imágenes PNG de SDL ENTONCES Mostrar error y salir

window <- Crear ventana ("Pac-Man") centrada con las medidas del mapa

renderer <- Crear motor de dibujo enlazado a la ventana con aceleración gráfica

SI la ventana o el renderizador fallan al crearse ENTONCES Mostrar error,
cerrar SDL y salir

Cargar archivo "pikachu.png" y convertirlo en textura de memoria

Cargar archivo "gengar.png" y convertirlo en textura de memoria

Llamar Función cargarMapa() (Lee el laberinto)

Llamar Función buscarElementos() (Ubica posiciones e inicia contadores)

Parte 2: El Ciclo de Juego y Cierre

Entrada: Las interacciones del jugador, el paso del tiempo y las funciones que controlan la lógica del juego.

Salida: Acciones visuales que experimenta el usuario y los cambios en la pantalla

INICIO (Parte 2)

MIENTRAS jugando sea VERDADERO HACER:

MIENTRAS el usuario realice una acción (teclado/ratón) HACER:

SI cierra la ventana o presiona ESC o la tecla 'X' ENTONCES jugando <- FALSO

SI presiona flecha ARRIBA ENTONCES velFila <- -1, velCol <- 0

SI presiona flecha ABAJO ENTONCES velFila <- 1, velCol <- 0

SI presiona flecha IZQUIERDA ENTONCES velFila <- 0, velCol <- -1

SI presiona flecha DERECHA ENTONCES velFila <- 0, velCol <- 1

FIN MIENTRAS

SI pasaron más de 120 ms ENTONCES Avanzar al siguiente cuadro de animación de Pikachu y Gengar

SI pasaron más de 300 ms ENTONCES Llamar Función moverFantasmas()

SI pasaron más de 180 ms ENTONCES Llamar Función moverJugador(velFila, velCol)

Llamar Función revisarColisiones()

SI vidas <= 0 ENTONCES:

Mostrar ventana de aviso: "Has perdido todas tus vidas."

jugando <- FALSO

SINO SI pellets <= 0 ENTONCES (Si se comió todos los puntitos):

SI nivel < TOTAL_NIVELES ENTONCES:

nivel <- nivel + 1 (Avanza de nivel)

Llamar Función cargarMapa()

Llamar Función buscarElementos()

Esperar 500 milisegundos (Pausa de transición)

SINO:

victoria <- VERDADERO

Mostrar ventana de aviso: "Felicidades, completaste todos los niveles."

jugando <- FALSO

FIN SI

FIN SI

Llamar Función actualizarTitulo(window) (Actualiza vidas y puntos en la barra)

Llamar Función renderizar(renderer) (Dibuja todo de nuevo)

Esperar 16 milisegundos (Estabiliza el juego a unos 60 fotogramas por segundo)

FIN MIENTRAS

Llamar Función guardarPuntaje() (Escribe en puntajes.txt)

Imprimir en consola si ganó o perdió junto con el puntaje final

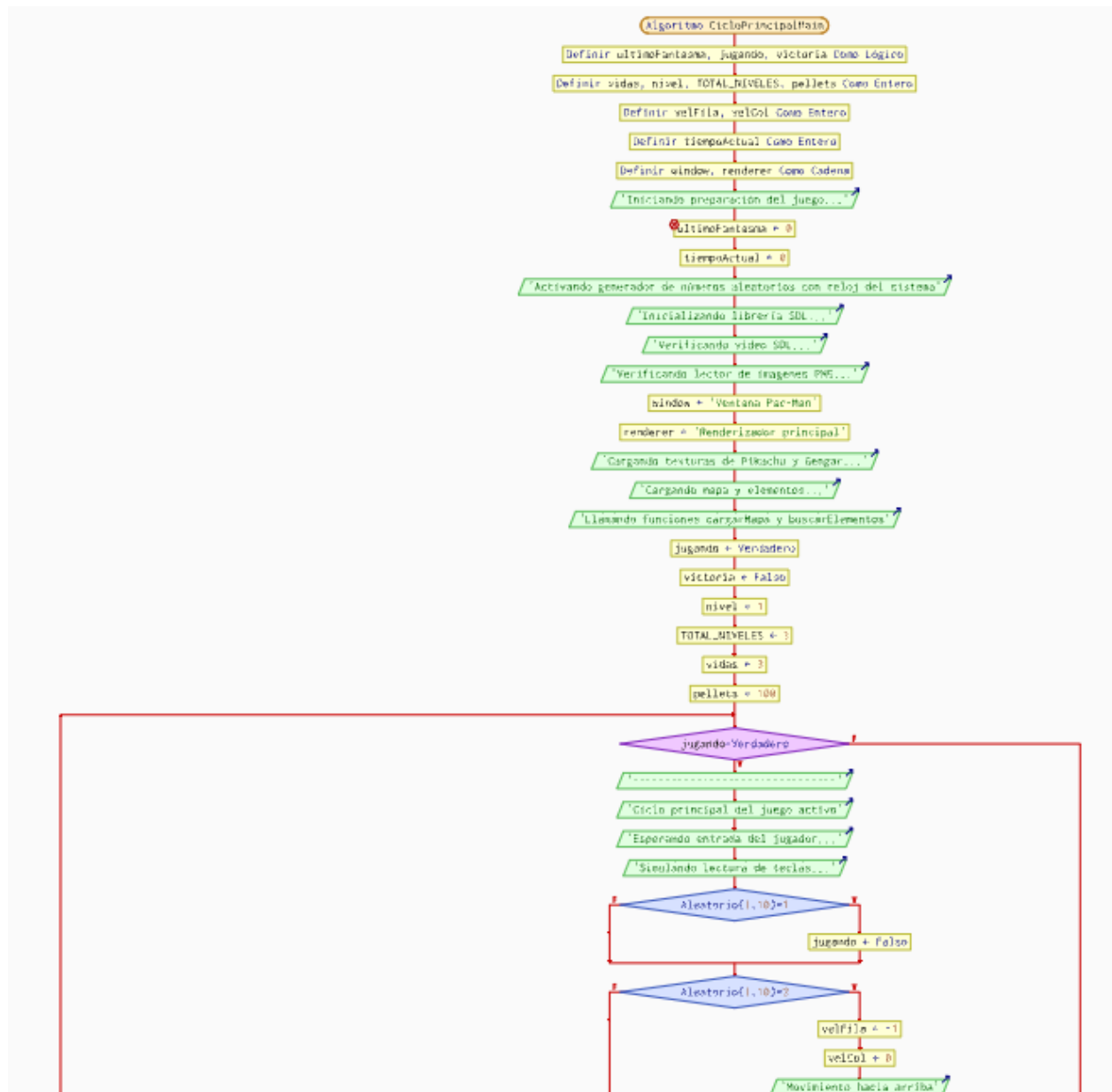
Destruir texturas de Pikachu y Gengar

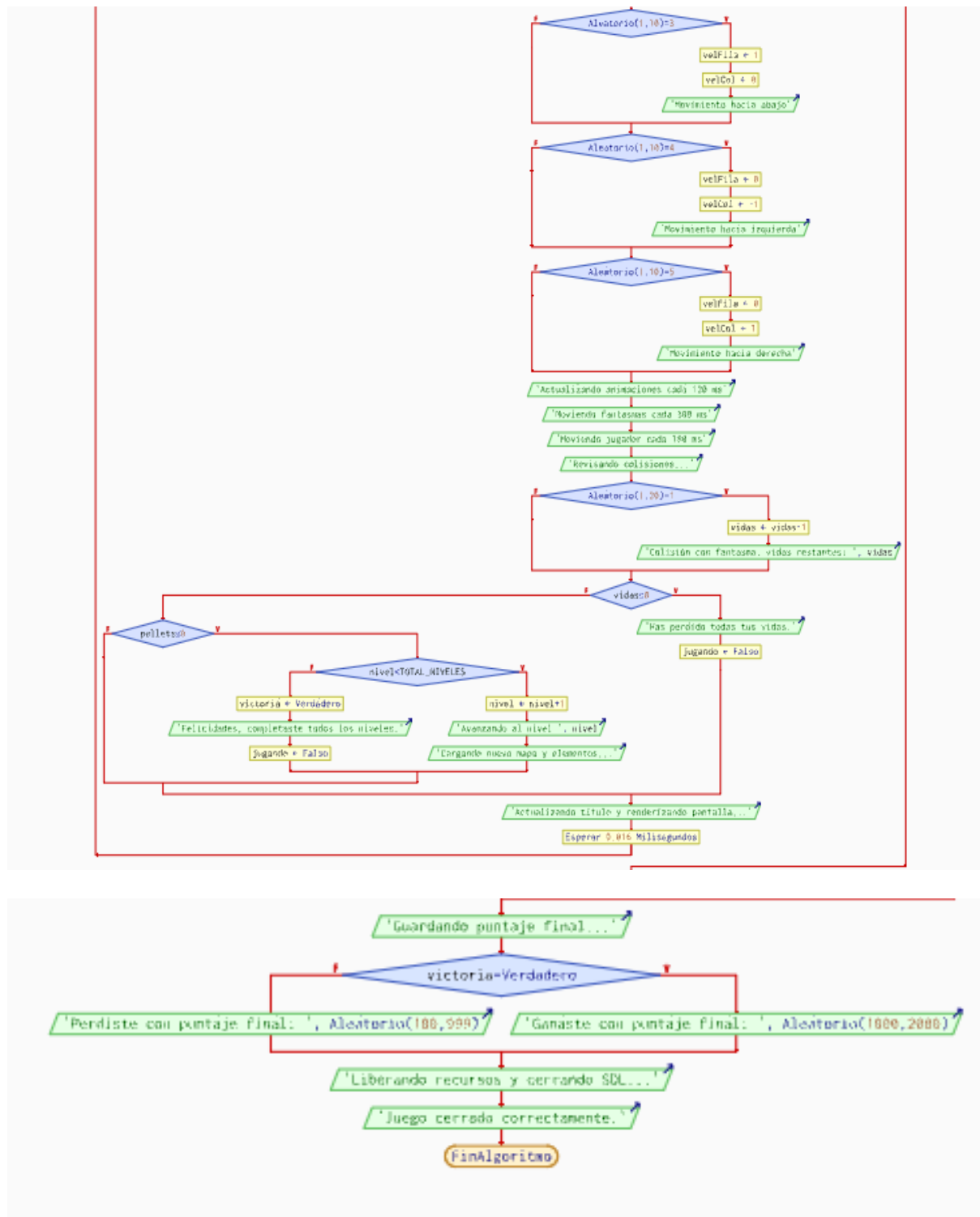
Cerrar lector de imágenes de SDL

Destruir renderizador y ventana principal

Cerrar por completo la librería SDL

FIN





Mover Jugador

Entrada: Dirección del movimiento, coordenadas actuales del jugador, matriz del mapa, puntos y pellets restantes

Salida: Nueva posición y dirección de la mirada

INICIO

nuevaFila <- jugadorFila + df (Calcula la fila del cuadro al que se quiere mover)

nuevaColumna <- jugadorCol + dc (Calcula la columna del cuadro al que se quiere mover)

SI nuevaFila se sale del mapa O nuevaColumna se sale del mapa ENTONCES

Terminar función (Evita que el juego se rompa por salirse de los límites)

FIN SI

SI mapa[nuevaFila][nuevaColumna] == "#" ENTONCES

Terminar función (Si hay una pared, no lo deja avanzar)

FIN SI

SI se mueve hacia arriba ENTONCES dirJugador <- 3 (Mirar arriba)

SI se mueve hacia abajo ENTONCES dirJugador <- 0 (Mirar abajo)

SI se mueve hacia la izquierda ENTONCES dirJugador <- 1 (Mirar izquierda)

SI se mueve hacia la derecha ENTONCES dirJugador <- 2 (Mirar derecha)

jugadorFila <- nuevaFila (Mueve oficialmente a Pikachu a la nueva fila)

jugadorCol <- nuevaColumna (Mueve oficialmente a Pikachu a la nueva columna)

SI mapa[nuevaFila][nuevaColumna] == "." ENTONCES

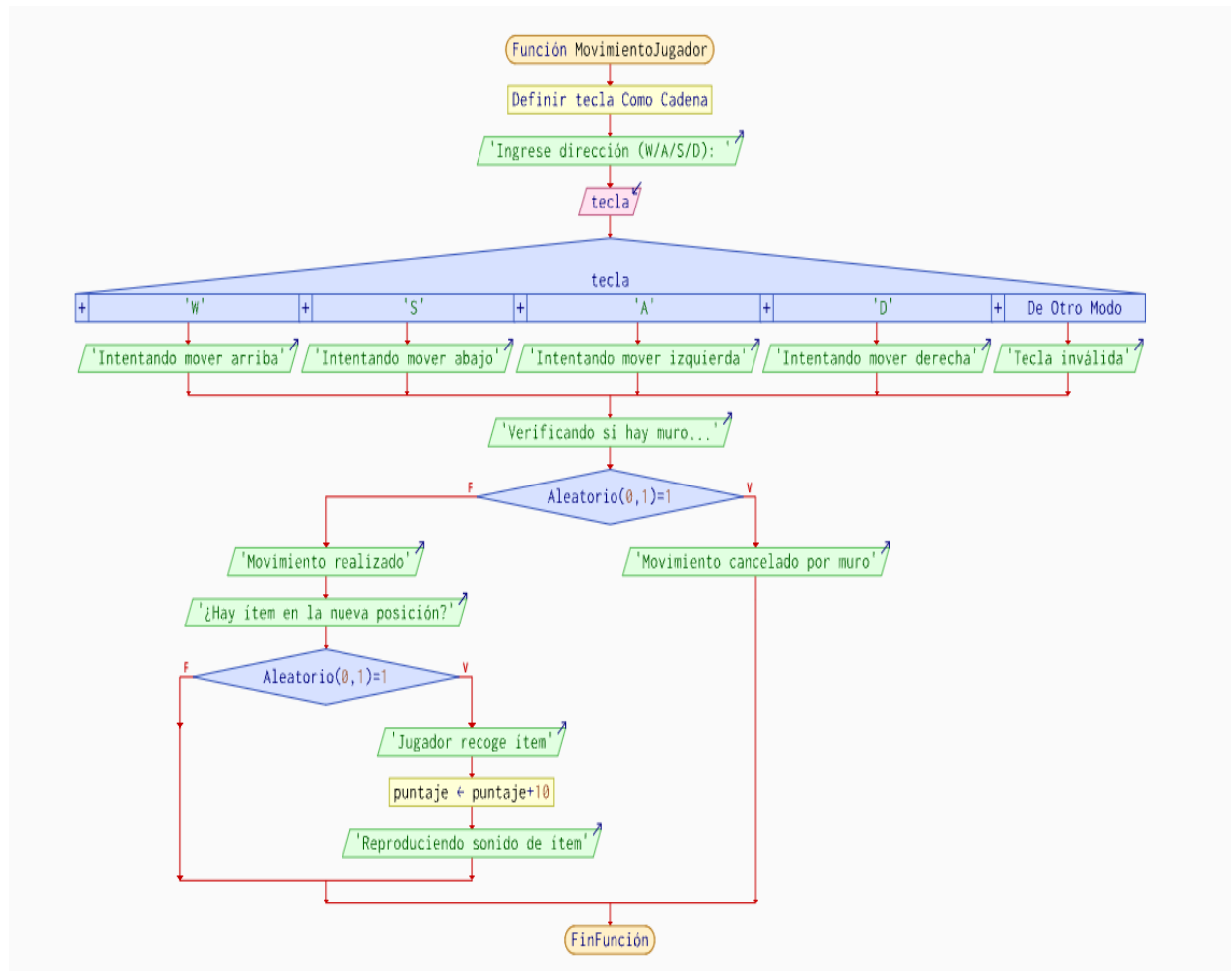
puntos <- puntos + 10 (Suma 10 puntos al marcador)

pellets <- pellets - 1 (Resta un puntito de la cuenta del nivel)

mapa[nuevaFila][nuevaColumna] <- " " (Borra el puntito comido del mapa)

FIN SI

FIN



Mover Fantasmas

Entradas: Mapa del juego, posición de Pikachu, posición de cada fantasma, número total de fantasmas y direcciones posibles.

Salidas: Nuevas posiciones de los fantasmas, dirección final del movimiento y la actualización visual del mapa.

INICIO

PARA $i \leftarrow 0$ Hasta $\text{totalFantasmas} - 1$ HACER:

SI un número al azar entre 1 y 10 es menor que 7 ENTONCES (70% de probabilidad de perseguir):

SI Pikachu está más a la izquierda ENTONCES dirección $\leftarrow$ izquierda

SINO SI Pikachu está más a la derecha ENTONCES dirección $\leftarrow$ derecha

SINO SI Pikachu está más arriba ENTONCES dirección $\leftarrow$ arriba

SINO SI Pikachu está más abajo ENTONCES dirección <- abajo

FIN SI

SINO (30% de probabilidad restante):

dirección <- Elegir dirección al azar entre las 4 posibles

FIN SI

PARA intento <- 0 Hasta 3 HACER (Buscar un camino libre):

nuevaFila <- fila actual del Gengar

nuevaColumna <- columna actual del Gengar

(Modificar la nueva fila o columna según la dirección elegida)

SI dirección es arriba ENTONCES nuevaFila <- nuevaFila - 1 y mirar arriba

SI dirección es abajo ENTONCES nuevaFila <- nuevaFila + 1 y mirar abajo

SI dirección es izquierda ENTONCES nuevaColumna <- nuevaColumna - 1 y mirar izquierda

SI dirección es derecha ENTONCES nuevaColumna <- nuevaColumna + 1 y mirar derecha

SI la nueva posición es un suelo libre y no choca con un muro ("#") ENTONCES:

fila actual del Gengar <- nuevaFila

columna actual del Gengar <- nuevaColumna

Terminar este bucle de intentos (Ya se pudo mover)

SINO (Si chocó con una pared):

dirección <- Elegir otra dirección al azar para el próximo intento

FIN SI

FIN PARA

FIN PARA

FIN

Revisar Colisiones

Entradas: Posición de Pikachu y de los fantasmas.

Salidas: Reduce vidas si hay colisión y reinicia posiciones.

INICIO

PARA $i \leftarrow 0$ Hasta $\text{totalFantasmas} - 1$ HACER:

SI $\text{jugadorFila} == \text{fantasmaFila}[i]$ Y $\text{jugadorCol} == \text{fantasmaCol}[i]$ ENTONCES

vidas $\leftarrow$ vidas - 1 (Te quitan una vida)

Llamar Función `reiniciarPosiciones()` (Todos vuelven a sus esquinas)

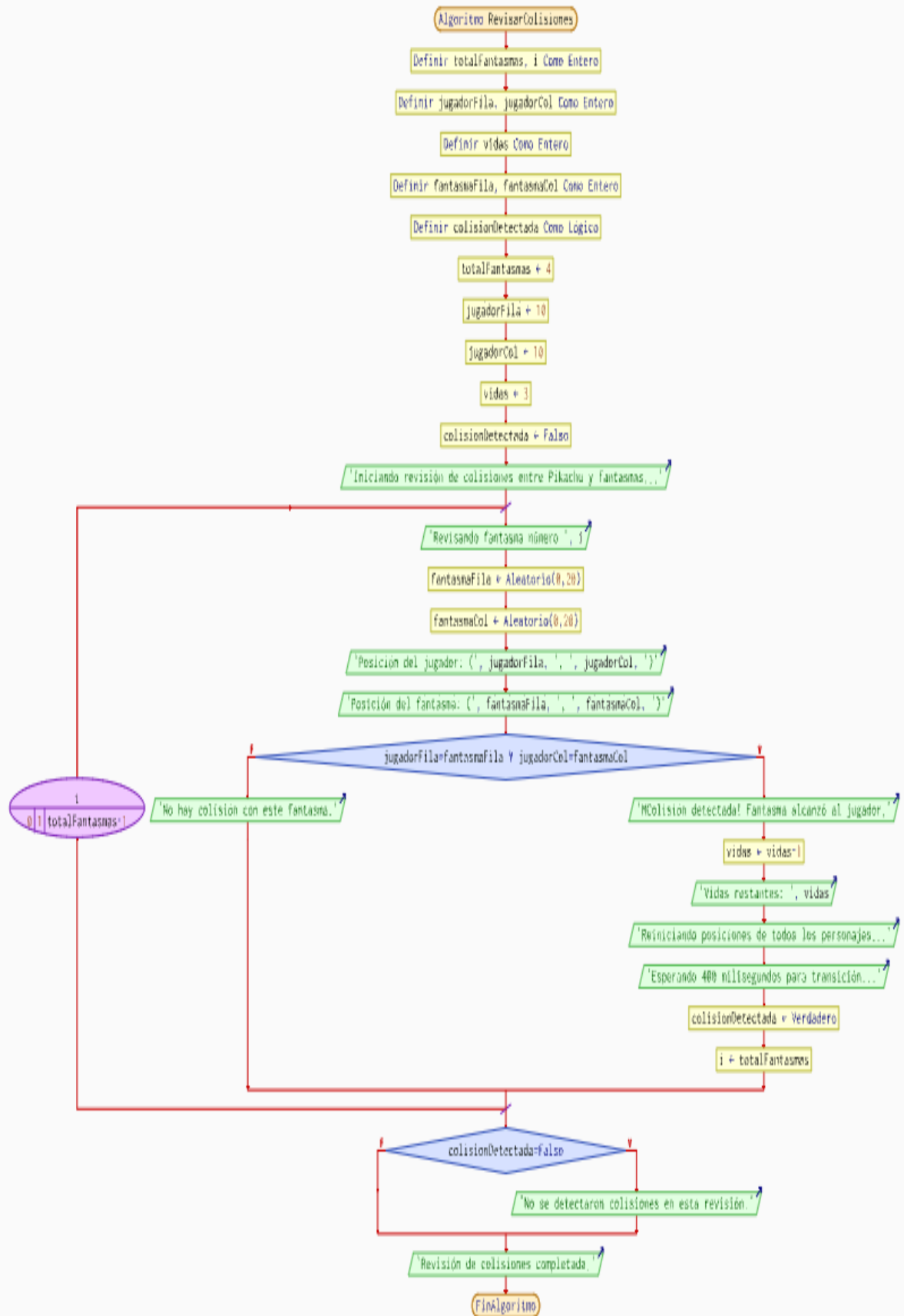
Esperar 400 milisegundos (Pausa técnica)

Terminar función (Salimos rápido para seguir jugando)

FIN SI

FIN PARA

FIN



Cargar Mapa

Entradas: Nivel actual y archivo del nivel.

Salidas: Llena la matriz del mapa con los datos del nivel.

INICIO

Generar el nombre del archivo según el nivel actual

Abrir archivo en modo lectura

PARA cada fila del mapa HACER

 SI el archivo existe y la fila puede leerse ENTONCES

 PARA cada columna de la fila HACER

 SI el carácter leído es un salto de línea o fin de cadena

ENTONCES

 Guardar un espacio en la matriz

SINO

 Guardar el carácter leído en la matriz

FIN SI

FIN PARA

SINO

PARA cada columna de la fila HACER

 Copiar el carácter correspondiente del mapa predeterminado

FIN PARA

FIN SI

 Colocar fin de cadena al final de la fila

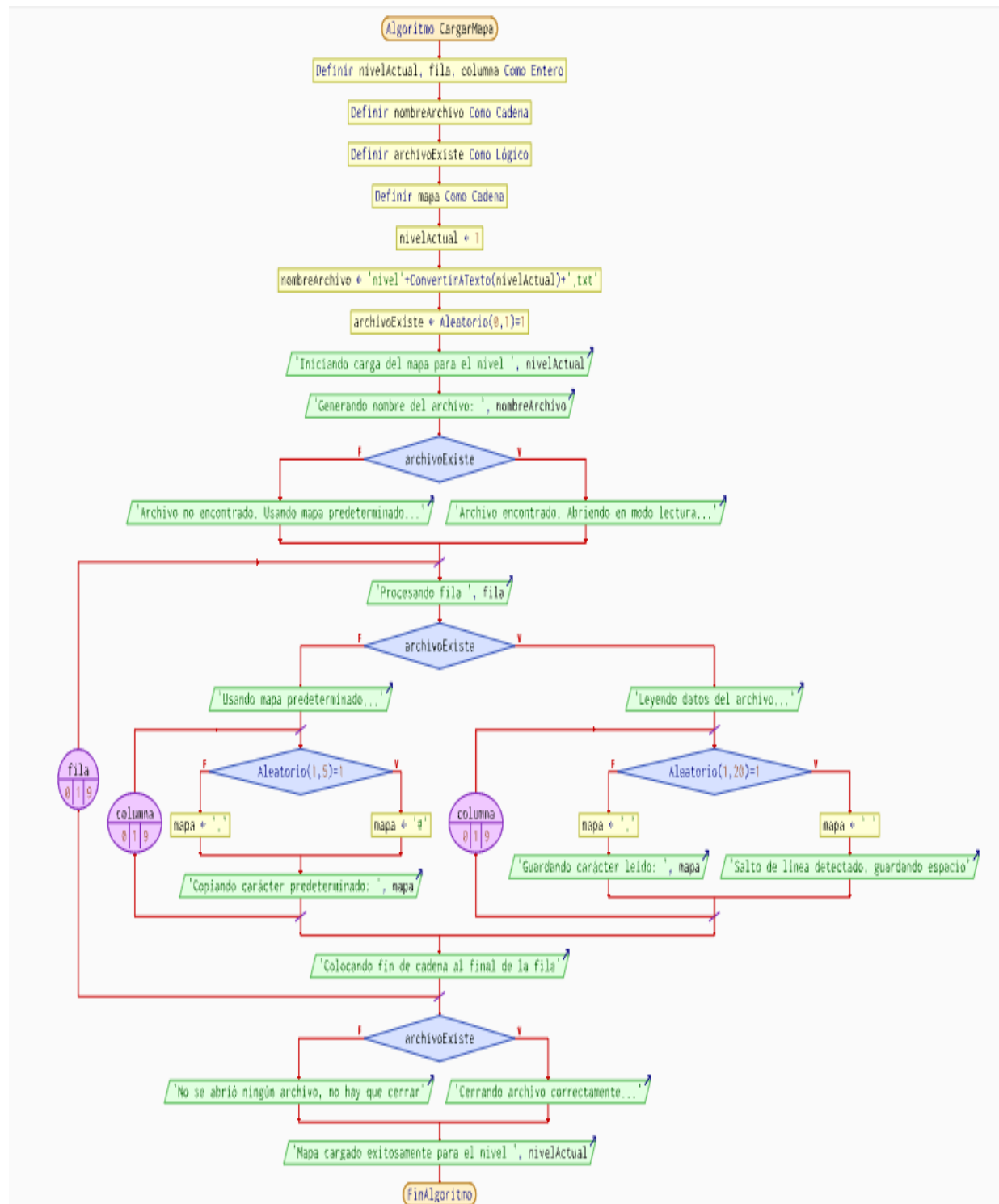
FIN PARA

SI el archivo fue abierto correctamente ENTONCES

 Cerrar archivo

FIN SI

FIN



Buscar Elementos

Entradas: Matriz del mapa.

Salidas: Obtiene posiciones iniciales, cuenta fantasmas y pellets.

INICIO

totalFantasmas <- 0 (Empezamos sin Gengars en la cuenta)

pellets <- 0 (Empezamos con cero puntitos en la cuenta)

PARA cada fila de la cuadrícula HACER:

PARA cada columna de esa fila HACER:

SI encontramos una 'P' ENTONCES:

Guardar la fila y columna actual como la posición de Pikachu.

Guardar esa misma posición como su punto de inicio por si muere.

Borrar la 'P' de la cuadrícula y dejar un espacio vacío " ".

SINO SI encontramos una 'F' (y aún no pasamos el límite de fantasmas)
ENTONCES:

Guardar la fila y columna actual para este Gengar.

Darle una dirección al azar para que empiece a moverse.

totalFantasmas <- totalFantasmas + 1 (Sumamos uno a la cuenta)

Borrar la 'F' de la cuadrícula y dejar un espacio vacío " ".

SINO SI encontramos un pellet '.' ENTONCES:

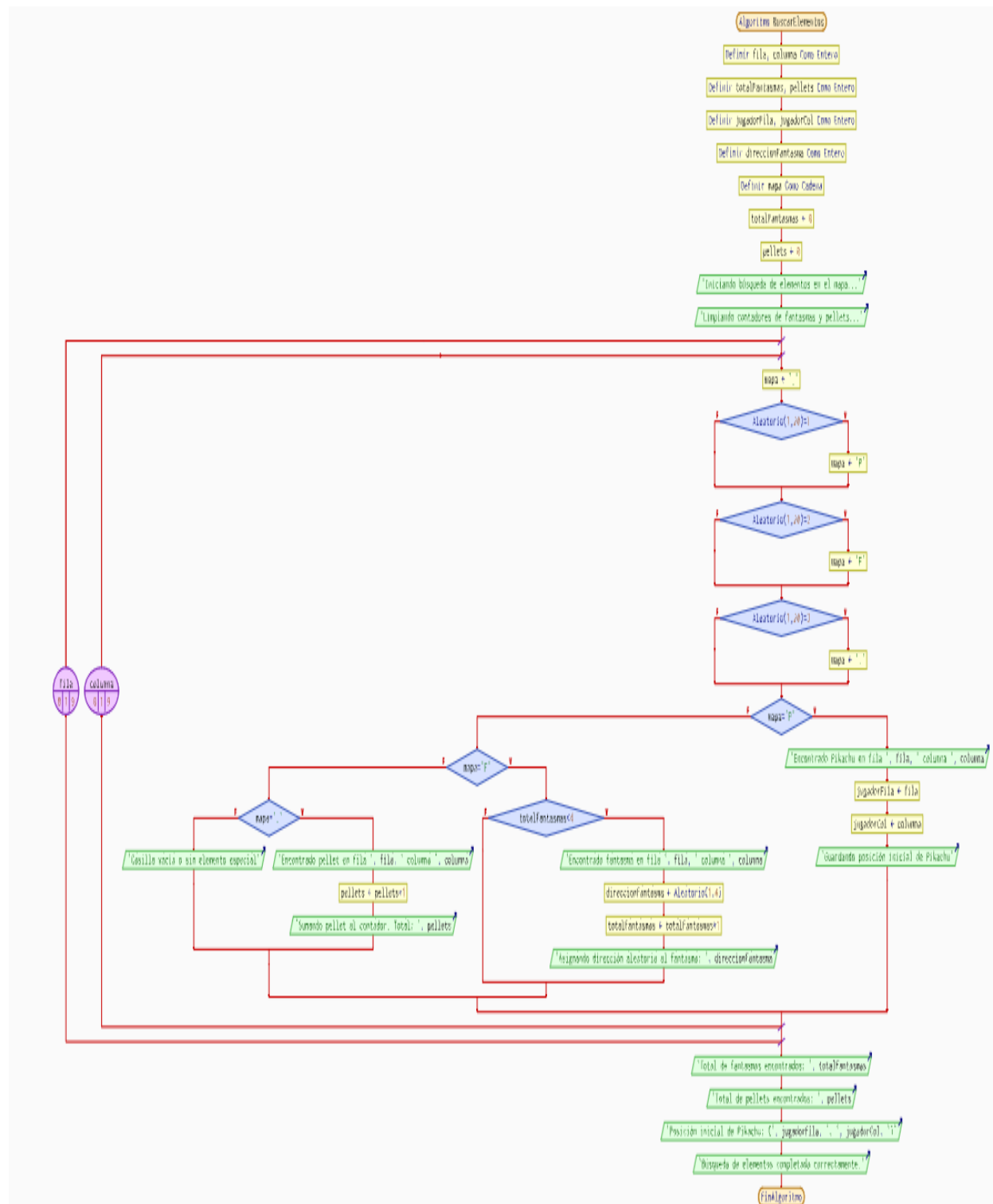
pellets <- pellets + 1 (Sumamos uno a la cuenta de comida)

FIN SI

FIN PARA

FIN PARA

FIN



Reiniciar Posiciones

Entradas: Coordenadas iniciales guardadas.

Salidas: Regresa a todos los personajes a su posición inicial.

INICIO

jugadorFila <- inicioJugadorFila (Pikachu regresa a su fila inicial)

jugadorCol <- inicioJugadorCol (Pikachu regresa a su columna inicial)

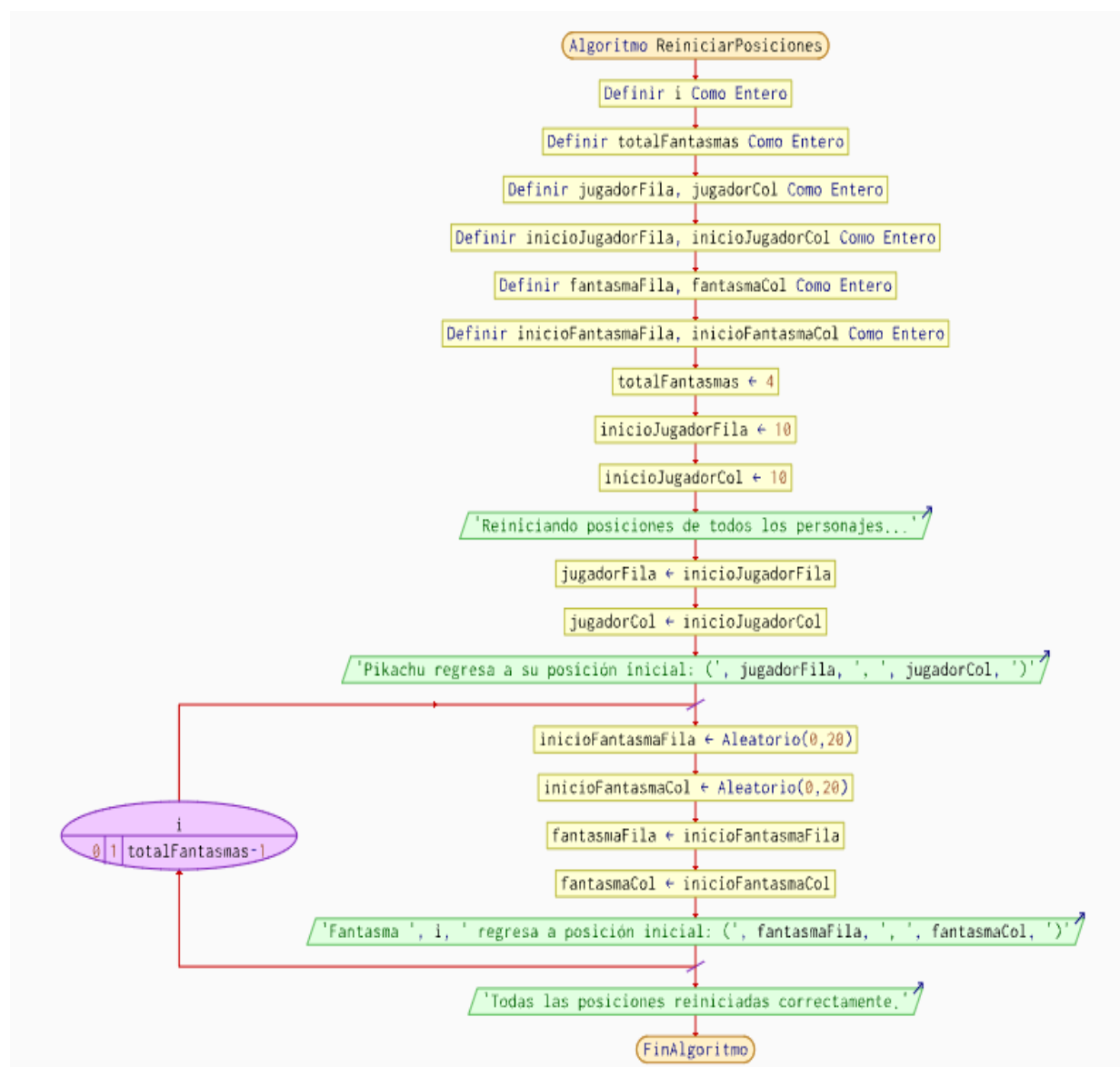
PARA i <- 0 Hasta totalFantasmas - 1 HACER:

fantasmaFila[i] <- inicioFantasmaFila[i] (El fantasma regresa a su fila inicial)

fantasmaCol[i] <- inicioFantasmaCol[i] (El fantasma regresa a su columna inicial)

FIN PARA

FIN



Guardar Puntaje

Entradas: Puntos, nivel y vidas.

Salidas: Guarda la información en puntajes.txt.

INICIO

archivo <- Abrir "puntajes.txt" en modo añadir ("a")

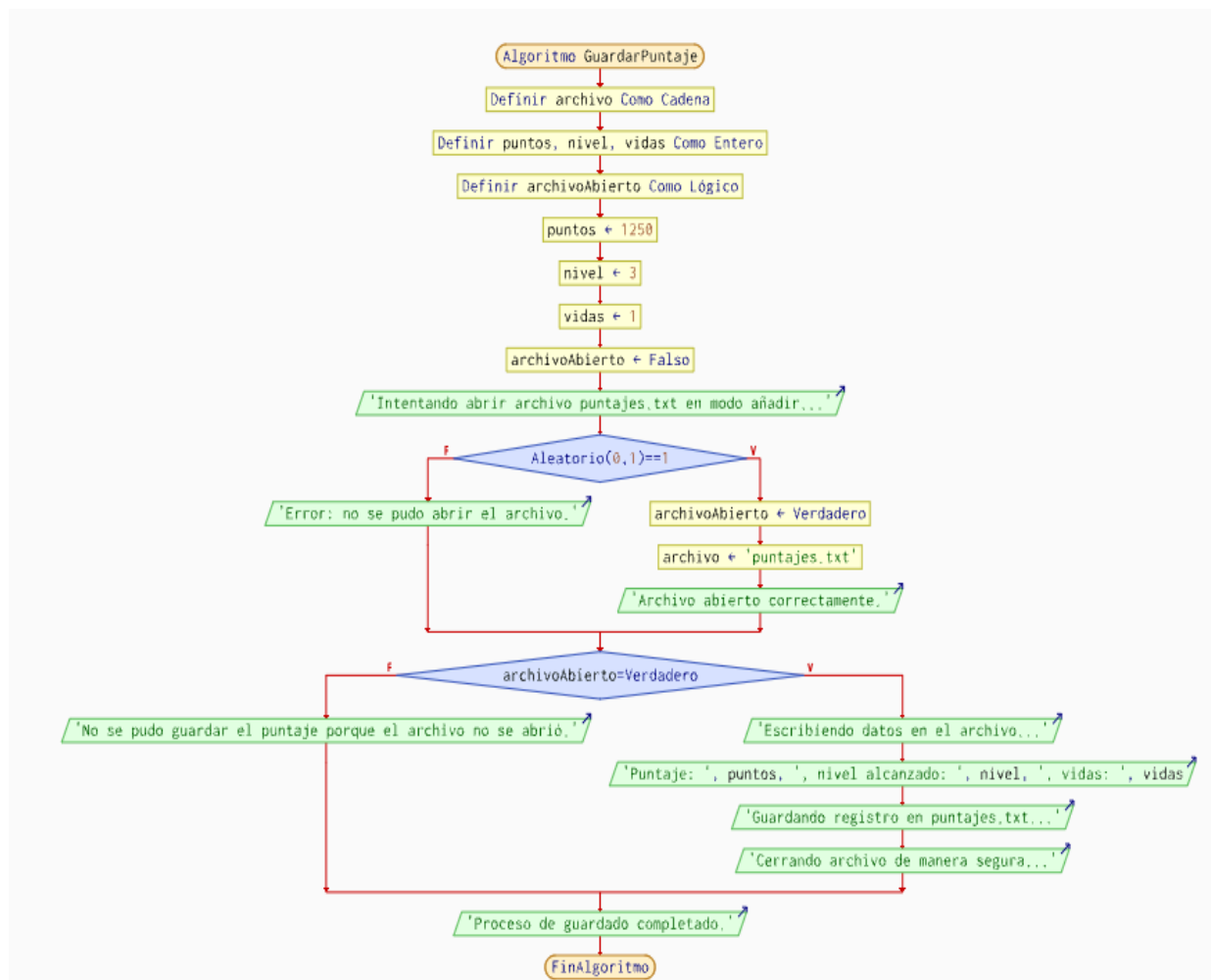
SI el archivo se abrió correctamente ENTONCES

Escribir en el archivo: "Puntaje: [puntos], nivel alcanzado: [nivel],
vidas: [vidas]"

Cerrar(archivo)

FIN SI

FIN



Renderizar

Entradas: Renderizador SDL, mapa, texturas y posiciones.

Salidas: Dibuja y actualiza todos los elementos en pantalla.

INICIO

Limpiar pantalla y pintarla de color Negro

PARA f <- 0 Hasta FILAS - 1 HACER:

PARA c <- 0 Hasta COLUMNAS - 1 HACER:

SI mapa[f][c] == "#" ENTONCES:

Dibujar un bloque relleno de color Azul en esa casilla

SINO SI mapa[f][c] == "." ENTONCES:

Dibujar un puntito relleno de color Amarillo/Crema en el centro de la casilla

FIN SI

FIN PARA

FIN PARA

pikachuDestino <- Calcular posición en píxeles basándose en su fila y columna actuales (con tamaño 48x48)

pikachuFrame <- Calcular qué cuadro exacto recortar de la hoja de imágenes según su animación y dirección

Dibujar a Pikachu en la pantalla combinando su recorte y su destino

PARA i <- 0 Hasta totalFantasmas - 1 HACER:

gengarDestino <- Calcular la posición en píxeles de este Gengar en la pantalla

gengarFrame <- Calcular qué cuadro recortar de su hoja de imágenes (tamaño 48x48)

Dibujar a este Gengar en la pantalla

FIN PARA

Mostrar y refrescar todo lo dibujado en la pantalla de juego

FIN

